

TIPOS DE CLASSES NA ORIENTAÇÃO A OBJETOS: ENTIDADE, *DTO* E SERVIÇO NA MODELAGEM DE SISTEMAS

Prof. Robson Calvetti

Resumo

A modelagem orientada a objetos permite representar sistemas de *software* por meio de classes que refletem elementos do domínio do problema e da estrutura da solução. No contexto da Engenharia de *Software*, diferentes tipos de classes desempenham papéis distintos, sendo fundamental compreender suas responsabilidades para a construção de modelos coerentes e bem organizados. Este artigo apresenta uma abordagem introdutória sobre os principais tipos de classes utilizados na modelagem de sistemas orientados a objetos: Entidade, *Data Transfer Object* – *DTO* e Serviço. São discutidas suas definições, características, aplicações e diferenças, bem como exemplos práticos no contexto de um sistema de atendimento baseado em filas. Ao final, destaca-se a importância da correta separação de responsabilidades para a qualidade do modelo e a evolução do sistema.

Palavras-chave: Orientação a Objetos; Modelagem de Classes; Entidade; *DTO*; Serviço; Engenharia de *Software*.

Abstract

Object-oriented modeling enables the representation of software systems through classes that reflect both the problem domain and the structure of the solution. In Software Engineering, different types of classes play distinct roles, making it essential to understand their responsibilities in order to build coherent and well-structured models. This paper presents an introductory approach to the main types of classes used in object-oriented modeling: Entity, Data Transfer Object – DTO, and Service. Their definitions, characteristics, applications, and differences are discussed, along with practical examples in the context of a queue-based service system. Finally, the importance of proper separation of responsibilities for model quality and system evolution is highlighted.

Keywords: Object-Oriented Modeling; Class Modeling; Entity; *DTO*; Service; Software Engineering.

1. Introdução

A modelagem de sistemas orientados a objetos tem como objetivo representar a estrutura interna do *software* de forma organizada, coerente e alinhada ao domínio do problema. Nesse contexto, o Diagrama de Classes desempenha papel fundamental ao definir classes, atributos, métodos e relacionamentos.

Entretanto, nem todas as classes possuem o mesmo papel dentro do sistema. A correta identificação do tipo de classe é essencial para garantir clareza na modelagem, facilitar a manutenção do sistema e evitar problemas como acoplamento excessivo e baixa coesão.

Dentre os tipos de classes mais utilizados, destacam-se as classes do tipo Entidade, *Data Transfer Object* – *DTO* e Serviço. Cada uma delas possui responsabilidades específicas e contribui de maneira distinta para a organização do sistema.

2. Classes na modelagem orientada a objetos

Na orientação a objetos, uma classe representa um conjunto de objetos que compartilham características e comportamentos comuns. As classes são responsáveis por definir:

1. Atributos, que representam o estado dos objetos; e
2. Métodos, que representam o comportamento.

Segundo *Sommerville (2019)*, a modelagem adequada de classes deve refletir o domínio do problema, evitando a criação de estruturas artificiais ou excessivamente técnicas que dificultem a compreensão do sistema.

Dessa forma, a classificação das classes por tipo permite organizar melhor o modelo, atribuindo responsabilidades claras e evitando sobreposição de funções.

3. Classe Entidade

A classe Entidade representa um elemento do domínio do problema, ou seja, algo que existe no contexto real do sistema sendo modelado.

3.1. Características

As principais características de uma Entidade são:

- Possui identidade própria;
- Representa um conceito do domínio;
- Armazena dados relevantes; e
- Pode existir independentemente do processamento.

3.2. Exemplos

No contexto de um sistema de atendimento em clínica, exemplos de entidades incluem:

- Paciente;
- Senha; e
- Fila.

Essas classes representam elementos reais do sistema e são fundamentais para a modelagem.

3.3. Papel na modelagem

As Entidades são responsáveis por representar o estado do sistema. Elas constituem a base do modelo e devem ser derivadas diretamente do domínio do problema.

4. Classe *DTO*

A classe *DTO* (*Data Transfer Object*) é utilizada para transportar dados entre diferentes partes do sistema, sem conter lógica de negócio.

4.1. Características

As principais características de um *DTO* são:

- Possui apenas atributos;
- Não contém regras de negócio;
- Não executa processamento ou validações complexas;
- É utilizado para comunicação entre camadas; e
- Simplifica a transferência de dados.

4.2. Exemplos

No sistema de atendimento, pode-se utilizar:

- SenhaDTO, contendo:
 - Número da senha;
 - Posição na fila; e
 - *Status*.

Esse objeto pode ser utilizado para exibir informações no painel ou na interface do usuário.

4.3. Papel na modelagem

Os *DTOs* permitem desacoplar diferentes partes do sistema, evitando que entidades sejam expostas diretamente e facilitando a comunicação entre componentes.

5. Classe Serviço

A classe Serviço é responsável por implementar as regras de negócio do sistema, coordenando ações entre diferentes classes.

5.1. Características

As principais características de uma classe Serviço são:

- Concentra lógica de negócio;
- Coordena interações entre entidades;
- Não representa diretamente um elemento do domínio;
- Possui baixo acoplamento com o estado.

5.2. Exemplos

No sistema de atendimento, pode-se definir:

- FilaService, responsável por:
 - Gerar senha;
 - Consultar posição;
 - Chamar paciente; e
 - Registrar ausência.

5.3. Papel na modelagem

As classes de Serviço organizam o comportamento do sistema, evitando que regras de negócio fiquem dispersas entre várias classes.

6. Comparação entre Entidade, *DTO* e Serviço

A distinção entre esses tipos de classes é fundamental para a qualidade da modelagem.

<u>Tipo de Classe</u>	<u>Função Principal</u>
Entidade	Representar o domínio
<i>DTO</i>	Transportar dados
Serviço	Executar regras de negócio

Essa separação contribui para:

- Melhor organização do sistema;
- Maior clareza na modelagem; e
- Facilidade de manutenção.

7. Aplicação no desenvolvimento de *software*

A utilização adequada desses tipos de classes impacta diretamente na qualidade do sistema.

Durante a modelagem:

- Entidades são identificadas a partir do domínio;
- Serviços são derivados das funcionalidades; e
- *DTOs* são definidos conforme necessidade de comunicação.

Segundo *Pressman e Maxim (2016)*, a organização adequada das responsabilidades no sistema contribui para a construção de soluções mais robustas e flexíveis.

8. Vantagens da separação de responsabilidades

A utilização de diferentes tipos de classes apresenta diversas vantagens:

- Melhora a organização do sistema;
- Reduz o acoplamento entre componentes;
- Aumenta a coesão das classes;
- Facilita manutenção e evolução; e
- Melhora a comunicação entre desenvolvedores.

Essa abordagem está alinhada com boas práticas de projeto de *software*.

9. Limitações e cuidados

Apesar de suas vantagens, o uso inadequado desses conceitos pode gerar problemas.

Entre os principais erros, destacam-se:

- Criar classes com múltiplas responsabilidades;
- Colocar lógica de negócio em *DTOs*; e
- Modelar entidades sem relação com o domínio.

Portanto, é importante aplicar esses conceitos com critério e coerência.

10. Conclusão

A compreensão dos diferentes tipos de classes na orientação a objetos é essencial para a construção de modelos de *software* claros, organizados e coerentes com o domínio do problema.

As classes do tipo Entidade, *DTO* e Serviço desempenham papéis complementares, permitindo separar responsabilidades e estruturar o sistema de forma adequada. Essa separação contribui para melhorar a qualidade da modelagem, facilitar a comunicação entre equipes e apoiar a evolução do sistema ao longo do tempo.

Dessa forma, o uso consciente desses tipos de classes representa um importante passo na formação de modelos mais robustos e bem estruturados na Engenharia de *Software*.

Referências

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **The Unified Modeling Language User Guide**. 2. ed. Boston: Addison-Wesley, 2005.

LARMAN, C. **Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**. 3. ed. Upper Saddle River: Prentice Hall, 2005.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.